

Documentation Technique

Plateforme de Gestion de Joueurs VR — Divrsitee | Stage BTS SIO SLAM | Imdad BOURAIMA | 2025

Informations generales

Projet	Plateforme web de gestion de joueurs pour jeux de realite virtuelle
Entreprise	Divrsitee — Paris (www.divrsitee.fr)
Type de stage	Stage de developpement full-stack — 6 semaines
Periode	Juillet — Aout 2025
Technologies	PHP 8.2, MySQL / MariaDB, HTML5, CSS3, Bootstrap 5, PDO, XAMPP
Gestion projet	Methode Agile — Trello
Auteur	Imdad BOURAIMA — BTS SIO SLAM — IRIS Paris

Contexte et objectifs

Divrsitee est une entreprise parisienne specialisee dans les serious games de realite virtuelle visant a sensibiliser les participants aux discriminations et aux inegalites sociales. Dans le cadre de ce stage de 6 semaines, la mission confiee etait de concevoir et developper une plateforme web complete permettant aux administrateurs de :

- Enregistrer les joueurs participant aux experiences immersives en realite virtuelle
- Gerer les comptes administrateurs avec un systeme de validation
- Consulter, filtrer, modifier et supprimer les joueurs inscrits
- Controler les acces via un systeme multi-roles securise

La plateforme repond a un besoin reel de l'entreprise : avant ce projet, la gestion des joueurs etait manuelle. Ce projet a permis d'automatiser et de centraliser toutes ces donnees.

Architecture Technique

Stack technologique

Couche	Technologie	Version	Role
Serveur local	XAMPP / Apache	8.2.12	Environnement de developpement local
Backend	PHP	8.2	Logique metier, sessions, securite
Base de donnees	MySQL / MariaDB	10.4.32	Stockage des utilisateurs et joueurs
Frontend	HTML5 / CSS3	-	Structure et mise en page des vues
Framework CSS	Bootstrap	5.3.3	Interface responsive et composants UI
Acces BDD	PDO (PHP)	-	Requetes preparees et securite SQL

Architecture de la base de donnees

La base de donnees **divrsitee** contient 3 tables distinctes correspondant aux 3 types d utilisateurs :

Table	Champs principaux	Description
joueurs	ID_joueur, Nom, Prenom, Adresse_email, Serious_game, Date_inscription	Joueurs participant aux experiences VR
admin	ID, Email, Mot_passe (hash bcrypt), statut (en_attente/valide/refuse)	Administrateurs pouvant gerer les joueurs
super_admin	ID, Email, Mot_passe (hash bcrypt)	Super administrateur unique pouvant gerer les admins

Schema SQL — Table joueurs

```
CREATE TABLE `joueurs` (  
  `ID_joueur` int(10) NOT NULL AUTO_INCREMENT,  
  `Nom` varchar(100) NOT NULL,  
  `Prenom` varchar(100) NOT NULL,  
  `Adresse_email` varchar(200) DEFAULT NULL,  
  `Serious_game` varchar(100) DEFAULT NULL,  
  `Date_inscription` date DEFAULT curdate(),  
  PRIMARY KEY (`ID_joueur`)  
  ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Schema SQL — Table admin

```
CREATE TABLE `admin` (  
  `ID` int(10) NOT NULL AUTO_INCREMENT,  
  `Email` varchar(200) NOT NULL,  
  `Mot_passe` varchar(100) NOT NULL, -- Hash bcrypt PHP  
  `statut` enum('en_attente','valide','refuse') DEFAULT 'en_attente',
```

```
PRIMARY KEY (`ID`)
```

```
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Fonctionnalites de la plateforme

Module	Fonctionnalite	Fichier(s) PHP	Acces
Authentification	Connexion securisee email + mot de passe	index.php, Traitement_connexion.php	Public
Authentification	Deconnexion et destruction de session	Deconnexion.php	Connecte
Enregistrement	Inscription d un nouveau joueur VR	enregistrement.php, traitement_enregistrement.php	Admin / Super Admin
Gestion joueurs	Liste, recherche, filtre par date	admin.php	Admin / Super Admin
Gestion joueurs	Modification d un joueur	modifier_joueur.php	Admin / Super Admin
Gestion joueurs	Suppression d un joueur	supprimer_joueur.php	Admin / Super Admin
Gestion admins	Liste de tous les administrateurs	super_admin.php	Super Admin uniquement
Gestion admins	Validation d un compte admin en attente	valider_utilisateur.php	Super Admin uniquement
Gestion admins	Suppression d un administrateur	supprimer_utilisateur.php	Super Admin uniquement
Gestion admins	Modification d un administrateur	modifier_utilisateur.php	Super Admin uniquement
Inscription admin	Formulaire d inscription admin (en attente)	inscription_admin.php	Public

Systeme de roles et securite des acces

La plateforme implemente un systeme de controle d acces base sur 3 niveaux de privileges :

Role	Droits	Protection PHP
Joueur	Aucun acces a l interface admin — inscription uniquement via formulaire public	N/A — pas de session
Admin	Consulter, ajouter, modifier, supprimer les joueurs. Pas acces a la gestion des admins.	isset(\$_SESSION['admin']) ou isset(\$_SESSION['super_admin'])
Super Admin	Tous les droits Admin + gestion complete des comptes administrateurs (valider, modifier, supprimer).	isset(\$_SESSION['super_admin']) avec verification BDD

Chaque page sensible commence par une verification de session. Exemple dans super_admin.php :

```
session_start();  
  
if (!isset($_SESSION['super_admin'])) {
```

```
header('Location: index.php');  
exit;  
}  
// Double verification en base de donnees  
$stmt = $pdo->prepare('SELECT * FROM super_admin WHERE Email = ?');  
$stmt->execute([$SESSION['super_admin']['Email']]);  
if (!$stmt->fetch()) { session_destroy(); header('Location: index.php'); exit; }
```

Module Authentification — Detail technique

Flux de connexion

Le processus de connexion dans **Traitement_connexion.php** suit les etapes suivantes :

- Reception des donnees POST (email + mot de passe)
- Destruction de la session existante pour eviter les conflits
- Verification en base : d abord super_admin, puis admin (ordre de priorite)
- Comparaison du mot de passe via **password_verify()** (hash bcrypt)
- Si super_admin : redirection vers super_admin.php
- Si admin valide : redirection vers accueil.php
- Si admin en attente : message d erreur, refus de connexion
- Si aucun compte : message d erreur generalise

Code — Verification super_admin

```
$stmt = $pdo->prepare('SELECT * FROM super_admin WHERE Email = ?');  
$stmt->execute([$email]);  
$super_admin = $stmt->fetch();  
  
if ($super_admin) {  
    if (password_verify($mot_passe, $super_admin['Mot_passe'])) {  
        $_SESSION['super_admin'] = $super_admin;  
        header('Location: super_admin.php');  
        exit();  
    } else {  
        $_SESSION['erreur'] = 'Mot de passe incorrect.';  
        header('Location: index.php');  
        exit();  
    }  
}
```

Securite des mots de passe

Les mots de passe sont hashes avec l'algorithme **bcrypt** via la fonction PHP **password_hash()** avec le cout par default (cost 10). La verification utilise **password_verify()** qui est resistant aux attaques par timing. Aucun mot de passe n'est stocke en clair en base de donnees.

Module Gestion des Joueurs — Detail technique

Affichage avec filtre et recherche (admin.php)

La page admin.php propose deux systemes de filtrage complementaires :

- **Filtre par date (cote serveur)** : parametre GET date_filtre, requete SQL avec WHERE
DATE(Date_inscription) = ?
- **Recherche par nom (cote client)** : JavaScript en temps reel, minimum 3 caracteres, filtre les lignes du tableau sans rechargement
- **Compteur dynamique** : mis a jour en JS pour afficher le nombre de joueurs correspondant au filtre actif

Code SQL — Requete avec filtre date

```
$sql = 'SELECT * FROM Joueurs';  
$params = [];  
  
if (!empty($date_filtre)) {  
    $sql .= ' WHERE DATE(Date_inscription) = ?';  
    $params[] = $date_filtre;  
}  
  
$sql .= ' ORDER BY Date_inscription DESC';  
$stmt = $pdo->prepare($sql);  
$stmt->execute($params);  
$joueurs = $stmt->fetchAll(PDO::FETCH_ASSOC);
```

Module Enregistrement des Joueurs

Formulaire d inscription (enregistrement.php)

Le formulaire d enregistrement collecte les informations suivantes pour chaque joueur VR :

Champ	Type HTML	Champ BDD	Obligatoire
Nom	text	Nom	Oui
Prenom	text	Prenom	Oui
Adresse e-mail	email	Adresse_email	Oui
Serious game teste	text	Serious_game	Oui
Date d inscription	date	Date_inscription	Non (default : aujourd hui)

Traitement de l insertion (traitement_enregistrement.php)

```
// Recuperation et nettoyage des donnees POST
$nom = htmlspecialchars(trim($_POST['nom']));
$prenom = htmlspecialchars(trim($_POST['prenom']));
$email = htmlspecialchars(trim($_POST['email']));
$jeu = htmlspecialchars(trim($_POST['serious_game']));
$date = !empty($_POST['date_inscription']) ? $_POST['date_inscription'] : date('Y-m-d');

// Insertion securisee via requete preparee PDO
$stmt = $pdo->prepare(
    'INSERT INTO joueurs (Nom, Prenom, Adresse_email, Serious_game, Date_inscription)'
    . ' VALUES (?, ?, ?, ?, ?)'
);
$stmt->execute([$nom, $prenom, $email, $jeu, $date]);

$_SESSION['message'] = 'Joueur enregistre avec succes !';
header('Location: enregistrement.php');
```

Connexion a la base de donnees (bdd.php)

La connexion a la base de donnees est centralisee dans le fichier **bdd.php** et utilise l extension PDO avec gestion des exceptions :

```
try {
    $pdo = new PDO(
        'mysql:host=localhost;dbname=divrsitee',
        'root',
        ''
    );
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    $pdo->exec("SET NAMES 'utf8'"); // Encodage UTF-8
```



```
} catch (PDOException $e) {  
die('Erreur de connexion : ' . $e->getMessage());  
}
```

L utilisation de PDO avec les requetes preparees (prepare() + execute()) protege systematiquement contre les injections SQL. Toutes les donnees utilisateur sont transmises en tant que parametres lies et non interpolees dans la requete SQL.

Interface Utilisateur

Pages et vues de l'application

Page	Fichier	Description	Rôle requis
Connexion	index.php	Formulaire de connexion avec logo Divrsitee	Public
Accueil	accueil.php	Page d'accueil avec les actions principales	Admin / Super Admin
Inscription joueur	enregistrement.php	Formulaire d'ajout d'un joueur VR	Admin / Super Admin
Gestion joueurs	admin.php	Tableau complet des joueurs avec filtres	Admin / Super Admin
Modifier joueur	modifier_joueur.php	Formulaire de modification d'un joueur	Admin / Super Admin
Gestion admins	super_admin.php	Tableau des administrateurs	Super Admin
Inscription admin	inscription_admin.php	Formulaire d'inscription admin (en attente)	Public
En-tête	en_tete.php	Header commun inclus dans toutes les pages	N/A — composant

Gestion des Administrateurs — Super Admin

Le Super Admin dispose d'un tableau de bord dédié (**super_admin.php**) lui permettant de gérer tous les comptes administrateurs. Les admins s'inscrivent via **inscription_admin.php** et leur compte est initialement en statut **en_attente** jusqu'à validation par le Super Admin.

Workflow de validation d'un admin

- 1. L'admin s'inscrit via **inscription_admin.php** (statut : **en_attente**)
- 2. Le Super Admin voit le compte dans **super_admin.php** avec le bouton Valider
- 3. Après validation : statut passe à valide — l'admin peut se connecter
- 4. Si refuse : statut refuse — la connexion est bloquée
- 5. Le Super Admin peut modifier ou supprimer un admin à tout moment

Code — Validation d'un admin

```
// valider_utilisateur.php
$id = $_POST['id'];
$stmt = $pdo->prepare(
    'UPDATE admin SET statut = ? WHERE ID = ?'
);
$stmt->execute(['valide', $id]);
```

```
header('Location: super_admin.php?message=admin_valide');
```

Les messages de confirmation (suppression, validation, modification) s'affichent via des paramètres GET et disparaissent automatiquement après 5 secondes grâce à un timer JavaScript.

Diagramme des fichiers et dependances

Fichier	Depend de	Inclus dans
index.php	Traitement_connexion.php	Point d entree public
Traitement_connexion.php	bdd.php	index.php (action du formulaire)
bdd.php	—	Tous les fichiers PHP backend
en_tete.php	style.css	Toutes les pages connectees
accueil.php	bdd.php, en_tete.php	Apres connexion Admin / Super Admin
admin.php	bdd.php, en_tete.php	Gestion joueurs
enregistrement.php	traitement_enregistrement.php	Ajout joueur
modifier_joueur.php	bdd.php	Modification joueur
supprimer_joueur.php	bdd.php	Suppression joueur
super_admin.php	bdd.php, en_tete.php	Gestion admins
valider_utilisateur.php	bdd.php	Validation admin
supprimer_utilisateur.php	bdd.php	Suppression admin
modifier_utilisateur.php	bdd.php	Modification admin

Bilan — Competences developpees

Competence	Description	Niveau
PHP Backend	Developpement de pages PHP dynamiques, gestion des sessions, redirections, includes	Operationnel
PDO / MySQL	Requetes preparees, CRUD complet, gestion des exceptions PDO	Operationnel
Securite web	Hachage bcrypt, protection XSS avec htmlspecialchars(), anti-injection SQL via PDO	Operationnel
Systeme de roles	Architecture multi-niveaux (Joueur / Admin / Super Admin) avec verification session	Operationnel
Bootstrap 5	Interface responsive, tableaux, formulaires, composants UI modernes	Operationnel
JavaScript (DOM)	Filtrage en temps reel, compteur dynamique, disparition des alertes avec setTimeout	Operationnel
Methode Agile	Utilisation de Trello pour la gestion des taches et le suivi du sprint	Applique
Travail en equipe	Communication avec le maitre de stage, respect des delais et livrables	Applique

Tests et validation

La plateforme a ete testee manuellement sur les scenarios suivants :

Scenario teste	Resultat attendu	Statut
Connexion avec email invalide	Message d erreur, pas de session	OK
Connexion admin en attente	Acces refuse, message explicite	OK
Connexion super admin valide	Redirection super_admin.php, session creee	OK
Ajout joueur avec tous les champs	Insertion BDD, message succes, redirection	OK
Filtre par date sans resultat	Tableau vide, message Aucun joueur	OK
Recherche par nom (min 3 caracteres)	Filtrage dynamique sans rechargement	OK
Suppression joueur avec confirmation	Suppression BDD, message rouge, redirection	OK
Validation admin par super_admin	Statut passe a valide, message vert	OK
Acces direct super_admin.php sans session	Redirection index.php, pas d acces	OK
Injection SQL dans le formulaire	Requete preparee : injection sans effet	OK

Conclusion et perspectives

Ce stage de 6 semaines chez Divrsitee m a permis de developper en conditions professionnelles une application web complete, fonctionnelle et securisee. La plateforme repond pleinement aux besoins exprimes par l entreprise : enregistrement des joueurs VR, gestion multi-roles et interface d administration intuitive.

Sur le plan technique, j ai consolide mes competences en PHP / MySQL, appris a mettre en place un systeme d authentication robuste avec bcrypt, et maitrise l utilisation de PDO pour securiser toutes les interactions avec la base de donnees.

Evolutions possibles

- Ajout d un tableau de bord avec statistiques (graphiques via Chart.js)
- Export des donnees joueurs en CSV ou Excel
- Systeme de notifications par email pour la validation des admins
- Migration vers une architecture MVC plus structuree (type Laravel)

- Ajout d'une API REST pour une future application mobile
- Mise en production sur un serveur distant avec HTTPS

Informations de contact et depot

Auteur	Imdad BOURAIMA
Email	bouraimasamory@gmail.com
GitHub	github.com/IMDAD1306/plateforme-gestion-joueurs
Formation	BTS SIO SLAM — IRIS Paris — 2025/2026
Encadrant	Divrsitee — Paris